The Connectomics Class: A High-Level Overview

The `Connectomics` class functions as a procedural 3D microcircuit builder. Its primary responsibility is to construct a biologically realistic, spatially embedded network of neurons (like a cortical column).

When initialized, it takes top-level volumetric constraints (e.g., layer depths, column radius, and raw cell densities for Excitatory/Inhibitory populations) and uses data from the Blue Brain Project (BBP) to:

- Generate thousands of specific individual neurons in 3D space (`cell_coords`).
- Assign each neuron a specific morphological type or "m-type" (e.g., a Layer 5 Tufted Pyramidal Cell).
- Wire these neurons together into a massive, sparse adjacency matrix based on distance-dependent connection probabilities.
- Calculate the specific number of physical synapses (multapses) formed for every established connection.

Detailed Breakdown: `get_lookup_table(self)`

1. Purpose

This function acts as a high-speed translator. When the simulation is handling tens of thousands of neurons, it constantly needs to know the basic biological properties of a cell based purely on its specific name. This function builds `self.mtype_fast_lookup`, an $O(1)$ dictionary that maps a highly specific cell string (e.g., `'L23_NBC'`) to its parent layer (`'L23'`) and its broad biological class (`'Inhibitory'`).

2. How it Works (Step-by-Step)

1. **The Acronym Map:** It begins by hardcoding a master dictionary called `acronym_map`. This maps standardized acronyms (like `'PC'` for Pyramidal Cell, or `'NBC'` for Nest Basket Cell) to either `'Excitatory'` or `'Inhibitory'`.
2. **Reading the Anatomy File:** It opens a text file named `S1-cells-distributions-Rat.txt` which contains the reference cell distributions from the BBP.
3. **Parsing the Data:** It reads the file line by line. For every line containing 5 columns (representing metype, mtype, etype, n, m), it isolates the mtype string.
4. **String Splitting:** It splits the mtype string at the first underscore (e.g., separating `'L23_LBC'` into `'L23'` and `'LBC'`). It includes special handling for Tufted Pyramidal Cells (TPC) to ensure their sub-layer variations are tracked correctly.
5. **Verification & Storage:** It cross-references the extracted acronym with the `acronym_map`. If the cell belongs to a valid layer (L1, L2/3, L4, L5, L6) and resolves to a valid biological type, it adds it to the dictionary.

3. The Output

The resulting dictionary is stored in `self.mtype_fast_lookup`. If you query `self.mtype_fast_lookup['L5_TT` it instantly returns `('L5', 'Excitatory')`.

Detailed Breakdown: `calculate_bbp_relative_presences(self)`

1. Purpose

While your `input_dict` tells the class how many total excitatory or inhibitory cells belong in a layer (e.g., "Put 10,000 excitatory cells in Layer 4"), it does not specify what kind of cells they should be. This function solves that by calculating the relative fraction of each specific m-type within its broader class, allowing the network to distribute those 10,000 cells realistically.

2. How it Works (Step-by-Step)

- **Step 1: Raw Parsing:** It opens the same `S1-cells-distributions-Rat.txt` file and extracts the 5th column (m), which represents the absolute total count of that specific m-type in the original reference model. It stores these raw numbers in a temporary dictionary (`mtype_raw_counts`).
- **Step 2: Grouping & Summing:** It iterates through those raw counts, splits the strings to identify the layer and biological class (again using the `acronym_map`), and bins them together.
  - It keeps a running tally of the absolute totals in `layer_totals` (e.g., the total sum of all L4 Excitatory cells in the reference model).
  - It keeps the individual m-type counts separated in a nested dictionary called `composition`.
- **Step 3: Calculating Percentages:** It loops through the composition dictionary. For every specific m-type, it divides its specific count by the total layer-class count calculated in Step 2. Equation used: perc $= (\text{count}/\text{total\_cells} \times 100)$.
- **Step 4: Debug Reporting:** It prints a highly detailed formatted console report, sorting the layers and displaying the exact calculated percentage and count for every modeled cell type in the circuit.

3. The Output

The final nested dictionary is stored in `self.bbp_results`. Later in the pipeline (specifically in `generate_microcolumn_cells`), the code will look at this dictionary to realize, for example, that if it needs to generate L2/3 Excitatory cells, exactly 55% of them need to be generated as Pyramidal Cells (PC) and the rest as other types based on these calculated biological fractions.

1. `get_ADJ(self)` (and its internal engines)

What it does:
This is an orchestrator function. `get_ADJ` (Get Adjacency Matrix) serves as the main trigger to physically build the spatial network. It guarantees that the neurons are placed in 3D space first, and then wires them together based on their distances and cell types.

How it works:
It executes two massive, computationally heavy functions in sequence:

1. `self.generate_microcolumn_cells()`:
    - It calculates the volume of each cortical layer based on the column radius and Z-boundaries.
    - Using the cell densities and the `bbp_results` percentages calculated earlier, it determines exactly how many of each specific cell type to create.
    - It generates random polar coordinates (r, θ) and a uniform Z-depth for each cell, resulting in the `self.cell_coords` array containing thousands of (x, y, z) locations.
2. `self.build_adjacency_matrix()`:
    - It groups all cells by their morphological type to avoid redundant calculations.
    - For every possible pathway (e.g., all `L23_PC` cells to all `L23_NBC` cells), it computes a pairwise Euclidean distance matrix (`cdist`).
    - It looks up the specific probability rule for that pathway (e.g., an exponential decay, a Gaussian curve, or step-bins) and calculates the exact probability of a connection for every single pair based on how far apart they are.
    - It "rolls the dice" (`np.random.rand() < p_sub`) to determine which connections actually form.
    - Finally, it stores all the successful connections in a highly compressed Sparse CSR Matrix (`self.adj_matrix`).

2. `extract_connectivity_dicts(self)`

What it does:
While a Sparse CSR Matrix is incredibly memory-efficient for mathematical operations, it is very slow and clumsy to query during a time-stepping neural simulation. This function translates the mathematical adjacency matrix into two highly optimized, easy-to-read Python dictionaries: one for incoming connections (afferences) and one for outgoing connections (efferences).

How it works (Step-by-Step):

- **Efferent Focus (`pre_to_post`):** The CSR (Compressed Sparse Row) matrix format is already optimized for reading rows. The function iterates through the matrix row by row (where each row is a pre-synaptic neuron). Using the matrix's internal pointers (`indptr` and `indices`), it instantly extracts a list of all column indices where a connection exists. This creates a dictionary where querying `pre_to_post[5]` returns a list of all post-synaptic neurons that neuron #5 sends signals to.
- **Afferent Focus (`post_to_pre`):** To find incoming connections quickly, reading rows is highly inefficient. Instead, the function converts the matrix into a CSC (Compressed Sparse Column) format. It then iterates column by column (where each column is a post-synaptic neuron). It extracts the row indices, creating a dictionary where querying `post_to_pre[10]` returns a list of all pre-synaptic neurons that fire into neuron #10.
- **Disk Saving:** Finally, if an output folder is specified, it uses `pickle.HIGHEST_PROTOCOL` to serialize and save these dictionaries to disk so they don't have to be recalculated in future simulation runs.

3. `extract_multapses(self)`

What it does:
Knowing who connects to whom is only half the battle. In the real brain, two connected neurons often form multiple physical synapses with each other (multapses). This function determines exactly how many synapses are formed for every single connection defined in the `post_to_pre` dictionary, and implements a biological gap-filling system for missing anatomical data.

How it works (Step-by-Step):

- **Data Parsing:** It reads the `synNumberperconex.dat` text file line-by-line, extracting the pathway name (e.g., `L4_PC:L5_NBC`), the mean number of synapses, and the standard deviation.
- **The Fallback Imputation:** Because the `.dat` file is often incomplete, the function groups the known data into broad macro-populations (e.g., all L4 Excitatory to L5 Inhibitory connections) and calculates a generalized average. This acts as a biological safety net for missing specific pathways.
- **Iterating the Network:** It loops through every post-synaptic neuron and grabs its array of pre-synaptic partners.
- **Assigning Statistics:** For each pre-synaptic partner, it tries to look up the exact mean and standard deviation for that specific pathway. If the data is missing, it seamlessly falls back to the macro-population average calculated earlier.
- **Vectorized Sampling:** Instead of running a random number generator one synapse at a time, it passes the entire array of means and standard deviations into `np.random.normal()`. This highly optimized NumPy operation draws the synapse counts for thousands of connections simultaneously.
- **Boundary Enforcement:** Because a Gaussian draw can result in numbers like 0.2 or -1.5, the function uses `np.maximum(1, np.round(sampled_synapses))`. This strictly enforces the rule: If the adjacency matrix says these two cells are connected, they must share at least 1 physical synapse.
- **Matrix Assembly:** It stacks the pre-synaptic IDs next to their generated synapse counts, creating an $N \times 2$ matrix for every post-synaptic cell, which is stored in `self.synapse_dict`.